CS-2710-1 Spring 2026

Computer Organization and Systems (COS)

Lecture #18

10 April 2026

Ashoka University

BBB

Control Hazards

A control hazard arises when the computation of the destination of a branch and update of PC are awaited, and hence, the instruction to be executed next, is uncertain at this moment

Let $t1 = $t2

beq $t1, $t2, Target IF ID/RR EX MEM WB

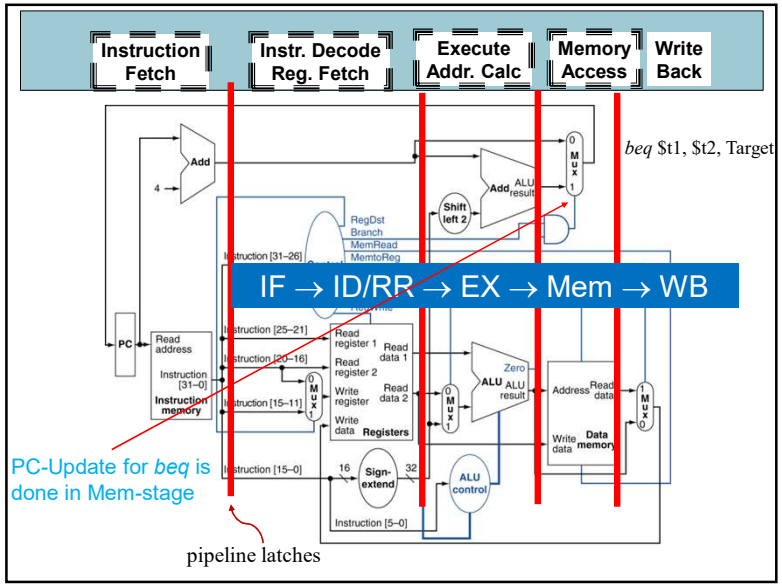
lw $t3, 0($t4) ~~IF ID/RR EX MEM WB~~

…

…

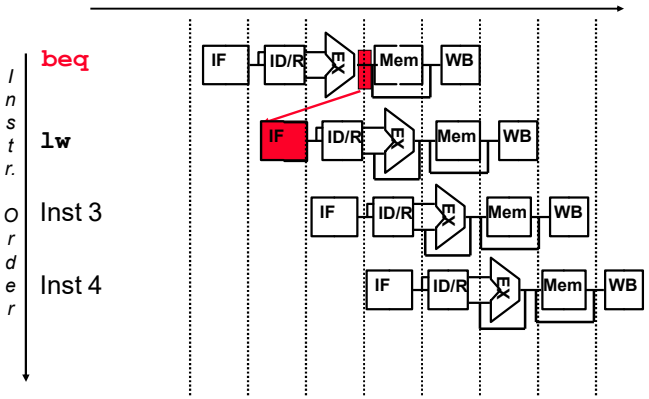
Target: add $t1, $t2, $t3

What will happen to the *lw* instruction?



Example: Control Hazards due to Branch

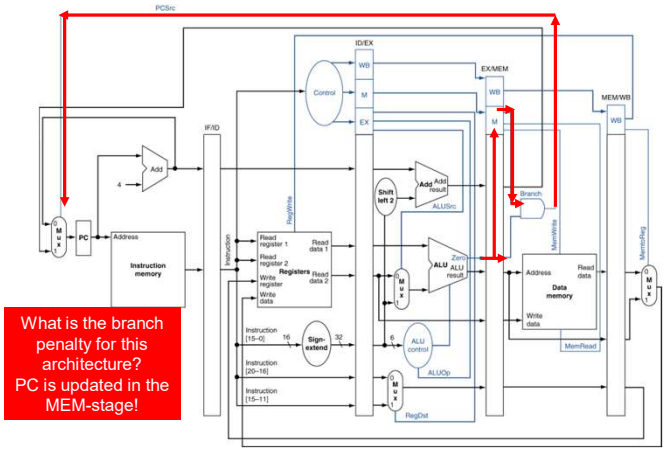
- Dependencies backward in time cause hazards



Branch Penalty

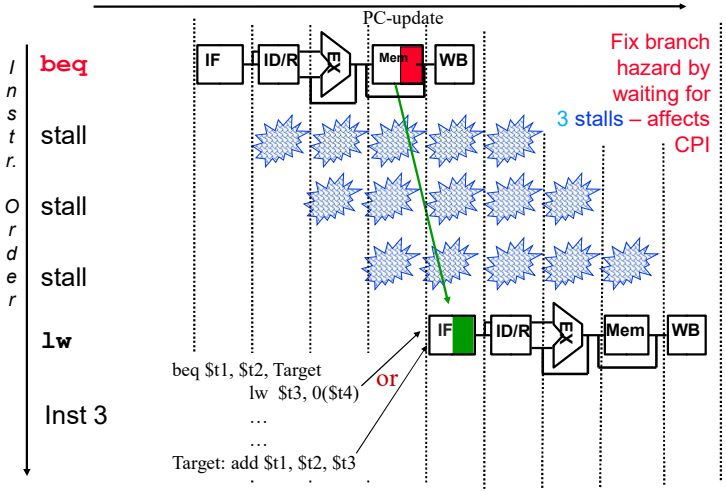
beq	IF	ID	EX	MEM	WB
lw	*	*	*	IF	ID	EX	MEM	WB

branch penalty: 3 clock cycles



What is the branch penalty for this architecture?
PC is updated in the MEM-stage!

One Way to "Fix" a Control Hazard: Insert stalls

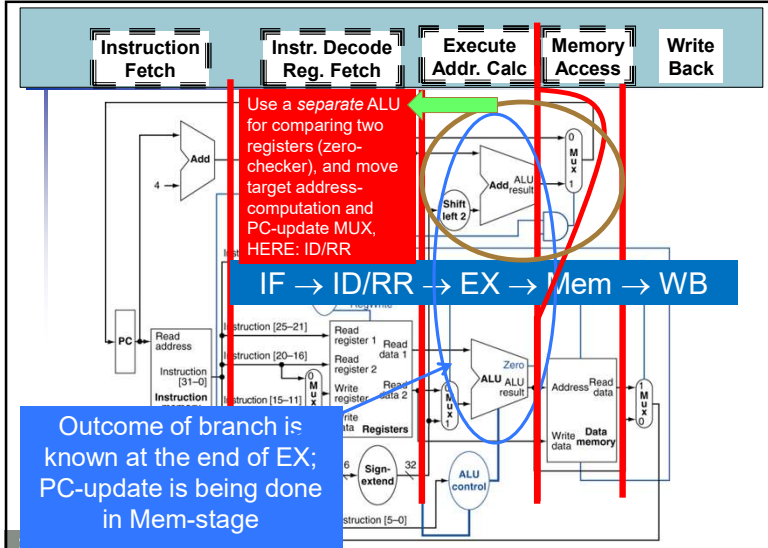


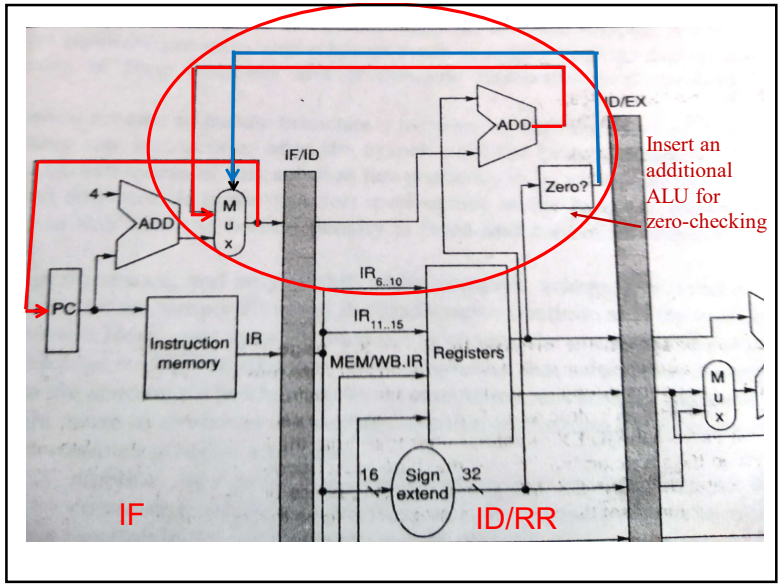
Control Hazards

- Hardware solution?

Can we compute branch target and PC-Update, earlier in the pipeline?

Would reduce stalls for successor instructions!



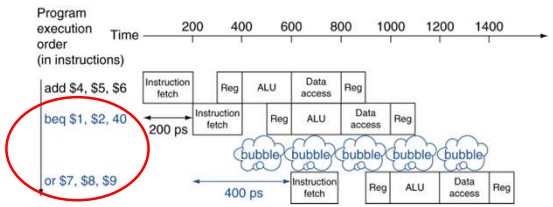


Control Hazards (*beq*)

- In MIPS pipeline, we can reduce branch penalty
  - by comparing registers (zero-checking) and computing target (PC-update) early in the pipeline
  - Add hardware to do it earlier in ID/RR-stage

Stall on Branch

- Wait until branch outcome determined (at ID/RR-stage) before fetching next instruction

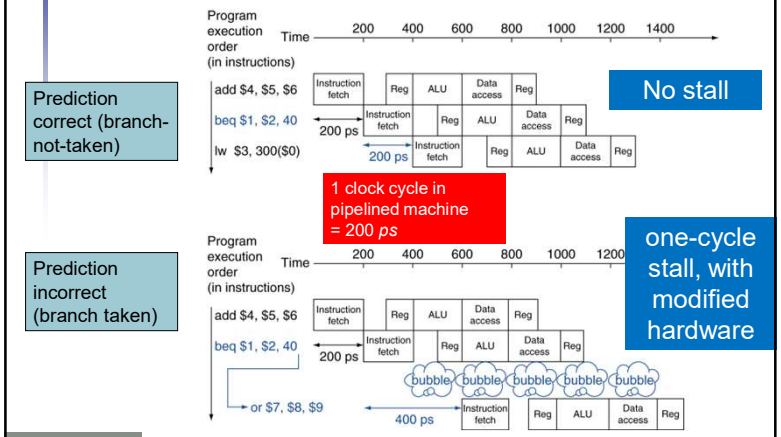


Instead of "three stalls", we now have only "one stall"

Another Approach: Branch Prediction

- Longer pipelines cannot readily determine branch outcome early
  - Fixed stall-penalty becomes unacceptable
- Predict outcome of branch
  - Stall only if prediction is wrong
- In MIPS pipeline, use
  - Predict-not-taken (normal flow: PC + 4)
  - Fetch instruction after branch, with no delay

MIPS with Predict-Not-Taken



Control Hazards

Branch Hazard Solutions

#1: Stall until branch target is known (3 stalls per branch)

#2: Move zero-checking (ZC) and target-address computation (TC) earlier (in ID/RR-Stage): Additional HW (one stall per branch; may conflict with preceding instr)

#3: Predict Branch-Not-Taken
- Execute successor instructions in sequence (no stall if branch is not taken)
- "Squash" instructions in pipeline if branch actually taken (one stall), with early ZCTC
- 47% MIPS branches not taken on average
- PC+4 already calculated, so use it to get next instruction

More Realistic Branch Prediction

- *Static* branch prediction
  - Based on typical branch behavior
  - Example: loop and if-statement branches
    - Predict backward branches taken
    - Predict forward branches not-taken
- *Dynamic* branch prediction
  - Hardware measures actual branch behavior
    - e.g., record recent history of each branch
  - Assume future behavior will continue the trend
    - When wrong, stall while re-fetching, and update history

Dynamic Branch Prediction

- In deeper and advanced pipelines, branch penalty is more significant
- Use dynamic prediction
  - Branch prediction buffer (*aka* branch history table)
  - Indexed by recent branch instruction addresses
  - Stores outcome (taken/not taken)
  - To execute a branch
    - Check table, expect the same outcome
    - Start fetching from fall-through or target
    - If wrong, flush pipeline and flip prediction

1-Bit Predictor

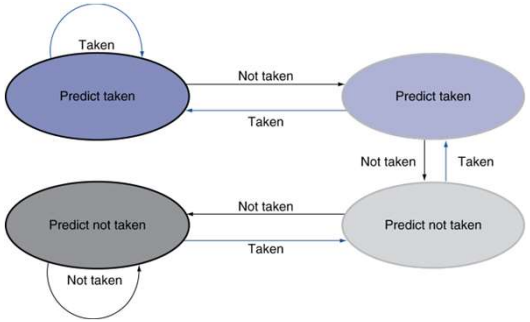
- Inner loop branches mispredicted twice!

```
outer: …
       …
inner: …
       …
       beq …, …, inner
       …
       beq …, …, outer
```

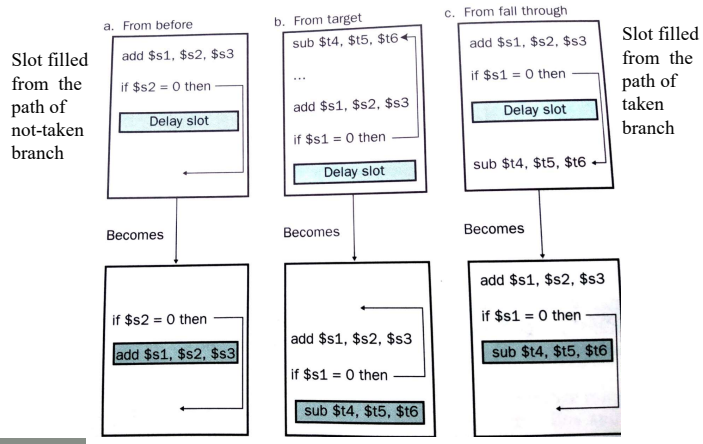
  - Mispredict as taken on last iteration of inner loop
  - Then mispredict as not taken on first iteration of inner loop next time around

2-Bit Predictor: Increases Prediction Accuracy

- Only change prediction on two successive mispredictions



Another solution to improve branch hazard performance: Delay-Slot (Compiler assisted)



Idea: Reorder code maintaining semantics, to buy time for branch to know the outcome

Stalls and Performance

The BIG Picture

- Stalls reduce performance
  - But are required to get correct results
- Compiler can arrange code to avoid hazards and stalls
  - Requires knowledge of the pipeline structure

Performance Equation

Pipeline overhead may degrade ideal speed-up

- Pipeline Speed-Up in an *ideal* machine
- = CPI_unpipelined / CPI_Pipelined
- ≈ Pipeline_Depth / CPI_Pipelined
- ≈ Pipeline_Depth

Ideal CPI_Pipelined = 1

- Pipeline Speed-Up in the presence of *stalls*
- = CPI_unpipelined / CPI_Pipelined
- ≈ Pipeline_Depth / (1 + Avg. #Pipeline_Stall_Cycles_per_Instruction)
- Example (Impact of branch penalty):
- Pipeline Speed-Up
- = Pipeline_Depth /{1 + (Branch_Freq. × Branch_Penalty)}

Hazards: Summary

To avoid structural hazard:
1. Use bubbles (stalls)
2. Use separate instruction/data memories or, separate instruction/data caches

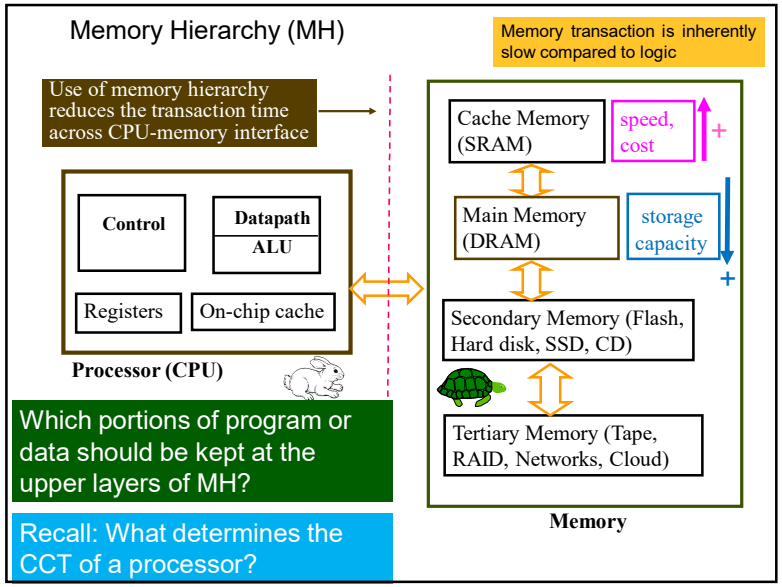
To avoid data hazard:
1. Use bubbles (stalls)
2. Use half-cycle Register Write (first)/Register Read (second)
3. Use data forwarding by hardware (EX/Mem → ALU_in; Mem/WB → ALU_in
4. Use code-reordering by compiler

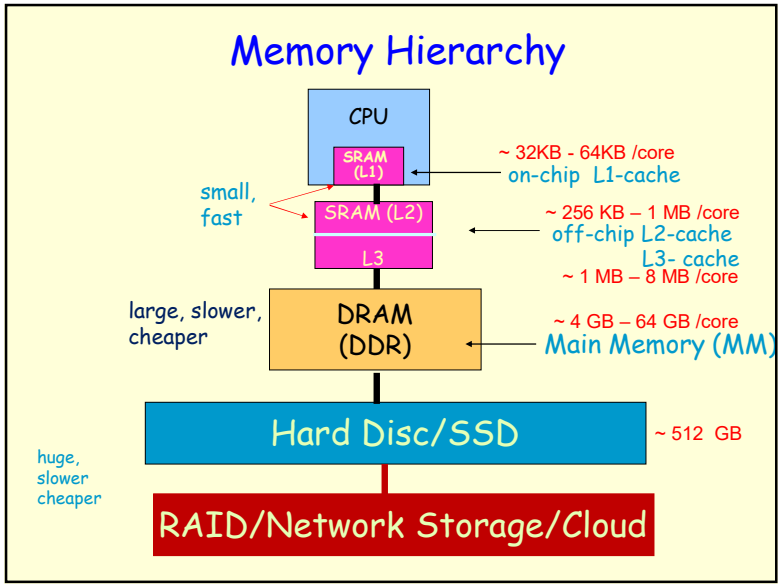
Pipeline Summary

- Pipelining improves performance by increasing instruction throughput
  - Executes multiple instructions in parallel
  - Each instruction has the same latency
- Subject to hazards
  - Structure, data, control
- Instruction set design affects complexity of pipeline implementation

Operating System (OS)

Process Management
Synchronization and Deadlocks
Memory Management
File System Management
I/O Management
Operating System Structures
Security and Protection





Principle of Locality

§5.1 Introduction

- Programs access a small proportion of their address space at any time
- Temporal locality
  - Items accessed recently are likely to be accessed again soon
  - e.g., instructions in a loop, induction variables
- Spatial locality
  - Items near those accessed recently are likely to be accessed soon
  - E.g., sequential instruction access, array data

Taking Advantage of Locality

- Memory hierarchy
- Store everything on disk
- Copy recently accessed (and nearby) items from disk to smaller DRAM memory
  - Main memory
- Copy more recently accessed (and nearby) items from DRAM to smaller SRAM memory (Cache)
  - Cache memory attached to CPU (on-chip cache)

Motivating Idea behind Memory Hierarchy

Keep most important items (Instructions/Data) close to you (CPU)

"close" means at higher levels of memory hierarchy, e.g., at cache (relative to MM); or at MM (relative to Disc storage), etc. ..

"important" refers to items that are requested by CPU frequently or expected to be requested soon ...

How to do I anticipate/identify "important" items?

* Temporal and Spatial locality ...

The principle of locality

Two types of locality (empirical rules):

- *Temporal locality* - near in time: we often access the *same* item again and again (e.g., instructions in a loop)
- *Spatial locality* - near in space/distance: our next access is often very close to recent accesses (e.g., data in matrices)

This sequence of memory addresses has both types of locality

1, 2, 3, 1, 2, 3, 8, 8, 47, 9, 10, 8, 8 .

spatial temporal non-local

What will happen at the first request by the CPU? (compulsory misses)
How do I manage my cache to honor temporal locality?
How do I utilize spatial locality?

How does HW manage cache?

Taking advantage of temporal locality:

Bring an item into cache whenever it is referenced;

If the cache is "full", kick out something that has not been used recently;

(keep a time-stamp for the cache item that is furthest in the past)

Taking advantage of spatial locality:

While serving a request from CPU, bring to cache a *block* of contiguous items (*cache-line*), not just the requested item (i.e., retrieve consecutive words from MM)

Needs suitable cache architecture to support this:

Multi-word cache

Cache Vocabulary

- cache hit: an access where the item is already in cache
- cache miss: an access where the item is not in cache
- cache block size or cache line size: the amount of data that gets transferred on a cache miss
- instruction cache (I-cache): cache that can only hold instructions
- data cache (D-cache): cache that can only hold data

Cache Issues

On a memory request,

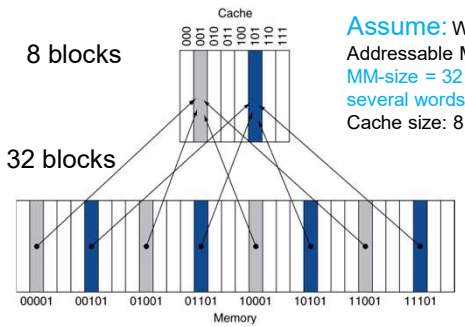
How does hardware know if it is a hit or miss in the cache?

On a cache miss,

- where to put the new item?
- which item to throw out to make room?
- how to remember which item is where?

Direct-Mapped Cache

- Location in cache determined by MM-address
- Direct mapped: MM → cache, is fixed, i.e., the destination in cache is unique;
  - (Block address) modulo (#Blocks in cache) Many-to-one mapping



Assume: Width of address bus: 5 bits
Addressable Memory Space = 2^5 = 32 blocks
MM-size = 32 blocks (a block might comprise several words);
Cache size: 8 blocks

- #Blocks is a power of 2
- Use low-order address bits;
- CPU generates MM-address;
- How it is converted to cache address (index)?

Does a cache entry contain garbage?

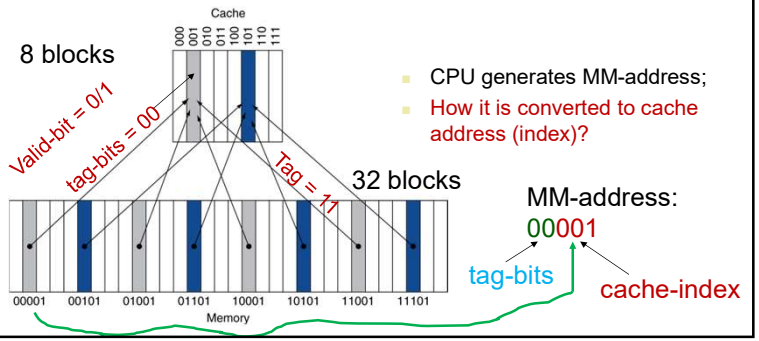
- Problem: when a program starts up, cache is empty (garbage).
  - It might contain data left from previous program/random bits;
  - How do you make sure you don't match a tag accidentally?
- Solution:
- An extra "valid" bit per cache-entry

Tags and Valid Bits

- How do we know which particular block is stored in a cache location?
  - Store missing part of block address as well as the data (cache data means instructions/data)
  - Actually, only need the high-order bits
  - Called the tag (resolves the problem of many-to-one mapping: MM→ Cache)
- What if there is no data in a location?
  - Valid bit: 1 = present, 0 = not present
  - Initially all valid-bits set to 0; a valid-bit is set to 1, when the requested item is loaded from MM to cache

Direct-Mapped Cache

- Location in cache determined by MM-address
- Direct mapped: only one choice
  - (Block address) modulo (#Blocks in cache)



- CPU generates MM-address;
- How it is converted to cache address (index)?

Cache Example

MM address requests:
22, 26, 22, 26, 16, 3, 16, 18

- 5-bit MM-address
- 8-blocks, 1 word/block, direct-mapped
- Initial state

Cache Index	Valid bit	Tag	Data
000	0		
001	0		
010	0		
011	0		
100	0		
101	0		
110	0		
111	0		

Cache Example

MM address requests:
22, 26, 22, 26, 16, 3, 16, 18

Word addr	Binary addr	Hit/miss	Cache block
22	10 110	Miss	110

Cache Index	V	Tag	Data
000	0		
001	0		
010	0		
011	0		
100	0		
101	0		
110	1	10	Mem [10110]
111	0		

Cache Example

MM address requests:
22, 26, 22, 26, 16, 3, 16, 18

Word addr	Binary addr	Hit/miss	Cache block
26	11 010	Miss	010

Cache Index	V	Tag	Data
000	0		
001	0		
010	1	11	Mem[11010]
011	0		
100	0		
101	0		
110	1	10	Mem[10110]
111	0		

Cache Example

MM address requests:
22, 26, 22, 26, 16, 3, 16, 18

Word addr	Binary addr	Hit/miss	Cache block
22	10 110	Hit	110
26	11 010	Hit	010

Cache Index	V	Tag	Data
000	0		
001	0		
010	1	11	Mem[11010]
011	0		
100	0		
101	0		
110	1	10	Mem[10110]
111	0		

Cache Example

MM address requests:
22, 26, 22, 26, 16, 3, 16, 18

Word addr	Binary addr	Hit/miss	Cache block
16	10 000	Miss	000
3	00 011	Miss	011
16	10 000	Hit	000

Cache Index	V	Tag	Data
000	1	10	Mem[10000]
001	0		
010	1	11	Mem[11010]
011	0	00	Mem[00011]
100	0		
101	0		
110	1	10	Mem[10110]
111	0		

18	10 010	Miss	010

Cache Example

MM address requests:
22, 26, 22, 26, 16, 3, 16, 18

Word addr	Binary addr	Hit/miss	Cache block
18	10 010	Miss	010

Cache Index	V	Tag	Data
000	1	10	Mem[10000]
001	0		
010	1	**10**	**Mem[10010]**
011	1	00	Mem[00011]
100	0		
101	0		
110	1	10	Mem[10110]
111	0		

New item is entered re-writing the previous one